

3D-Printed AI-Powered Microscope and Cellular Imaging Studio

User Manual v0.1



Developers

Addison “Danny” Drobny, Dylan Lee, Kirby Burke, Mina Li, and Roberto Lucatero

Affiliation

UC Irvine Departments of Information & Computer Science (ICS) and
Electrical Engineering and Computer Science (EECS)

Table of Contents

Introduction	2
Features	2
Requirements	3
Hardware Requirements	3
Software Requirements	3
Installation.....	4
Cellular Imaging Studio Walk-Through.....	7
View Module.....	7
Move Module.....	8
Capture Module.....	9
Gallery Module	11
Logging Module.....	12
Settings Module.....	13
About Module.....	15
Exit Module	16
Troubleshooting	17
MIT License	18
Appendix: Software Architecture	19
Backend Python Libraries.....	19
Frontend Stack	19

Introduction

For our Senior Capstone Project, we fabricated a 3D-printed microscope and developed the Cellular Imaging Suite program – based on the [OpenFlexure Project](#) – to provide a low-cost solution for students, educators, and research professionals to capture high resolution cellular imagery, and leverage the added features of cell counting and motion tracking, with robust reporting.

Features

TO-DO: Write after confirming the features available in the final project build.

TO-DO: We have a "streaming" option in the Settings module. So, we need to write about it here. Is "streaming" an alternate connection option, way to navigate through the GUI, or what?

Requirements

The following resources are required or recommended to build and use the microscope and Cellular Imaging Studio program.

Hardware Requirements

- Assembled [OpenFlexure High-Resolution Microscope](#)
 - [Achromatic lens](#) (optional)
 - [40X plan objective lens](#) (optional)
- **Windows** PC with an Ethernet (i.e., RJ45) port or adapter
- Ethernet cable

Software Requirements

TO-DO: Add [guesstimated] system requirements and resources the user needs to build and install the app (core requirements – specific resources are listed in the Appendix), and communicate with the OpenFlexure server, if any.

Installation

Can we put the following workflow into an installer script to streamline this process?

1. Copy the repo
2. Run the installer script, which does all of these:
 - a. Installs Python dependencies (VS Code does this automatically from requirements.txt – for Python libraries at least)
 - b. Downloads other dependencies (e.g., FFmpeg)
 - c. Builds the Python server
 - d. Builds the Vue app
 - e. Consolidates and builds the app
3. Launch the app and connect the microscope.

Also, assert destination directories and installer filenames (e.g. '%LOCALAPPDATA%\Programs\ml-microscope' and 'setup_v1-1.exe')

Download installer instructions

First, build the installer, then install the full desktop application.

Step 1: Build the Frontend Vue Distribution

In terminal, move to directory

```
C:...\ml-microscope\src\microscope-frontend
```

Then run command

```
npm run build
```

Step 2: Build the Backend Python Server

Move back to root

```
C:...\ml-microscope
```

Download [ffmpeg.exe](#) to

```
C:...\ml-microscope\vendor\ffmpeg
```

Note: you might need to make this directory before downloading.

From root run command

```
py -m PyInstaller --clean --noconfirm --onedir --name  
MicroscopeBackend backend\client.py `--add-binary  
"vendor\ffmpeg\ffmpeg.exe;."
```

Step 3: Copy the Vue Distribution into the Python Server Folder

Run command:

```
xcopy /E /I /Y src\microscope-frontend\dist  
dist\MicroscopeBackend\frontend_dist
```

If you are rebuilding the installer instead run the commands:

```
rmdir /S /Q dist\MicroscopeBackend\frontend_dist 2>$null  
xcopy /E /I /Y src\microscope-frontend\dist  
dist\MicroscopeBackend\frontend_dist
```

The `rmdir` command might error; this is fine.

Step 4: Build and Run the App Installer

Move to desktop-app dir

```
C:...\ml-microscope\desktop-app
```

Then run the command

```
npm run dist
```

Step 5: Run Installer

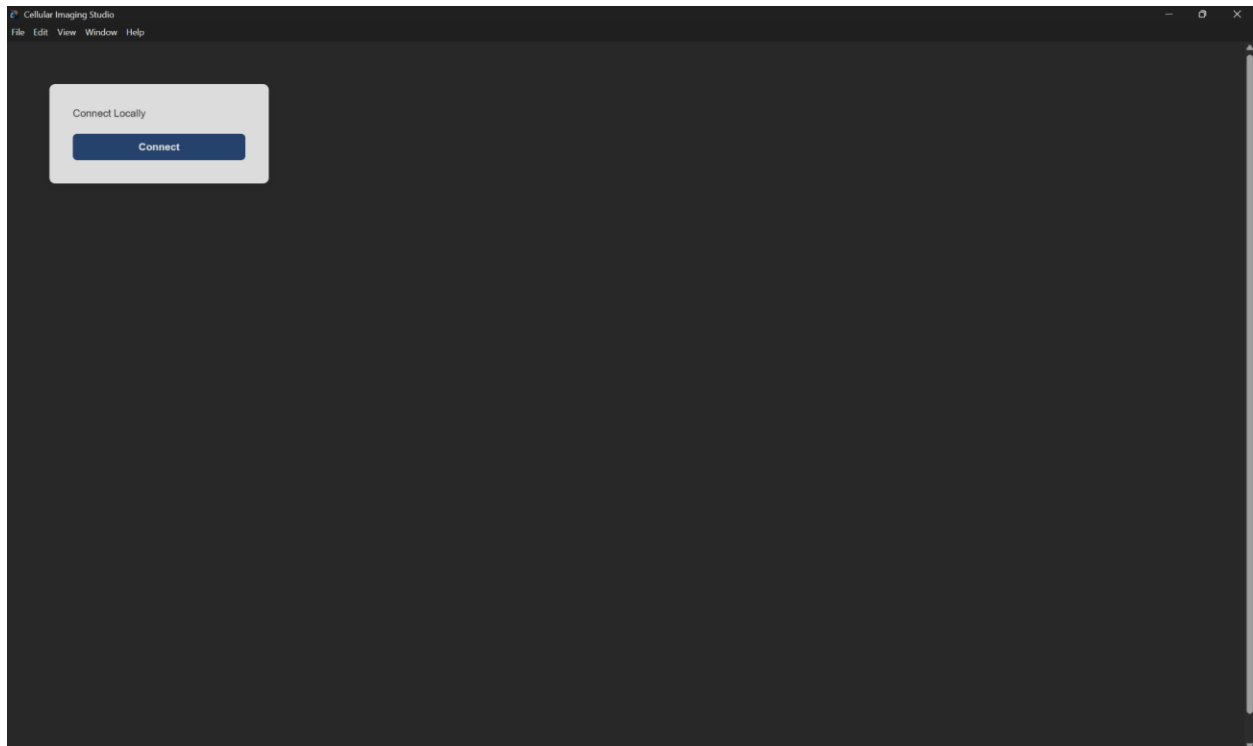
Find the copied repo in user files or where it was saved. The Cellular Imaging Studio Setup {v_number}.exe installer will be in:

```
C:...\ml-microscope\desktop-app\release-build
```

Run Cellular Imaging Studio Setup {v_number}.exe

Step 6: Connect to Microscope

Prior to connecting to the microscope from the Cellular Imaging Studio program, plug the microscope into a stable power source and connect it to your computer with an Ethernet cable. The microscope may take several minutes to power up. So, please be patient. The microscope's port LEDs will blink green when it's ready to connect.



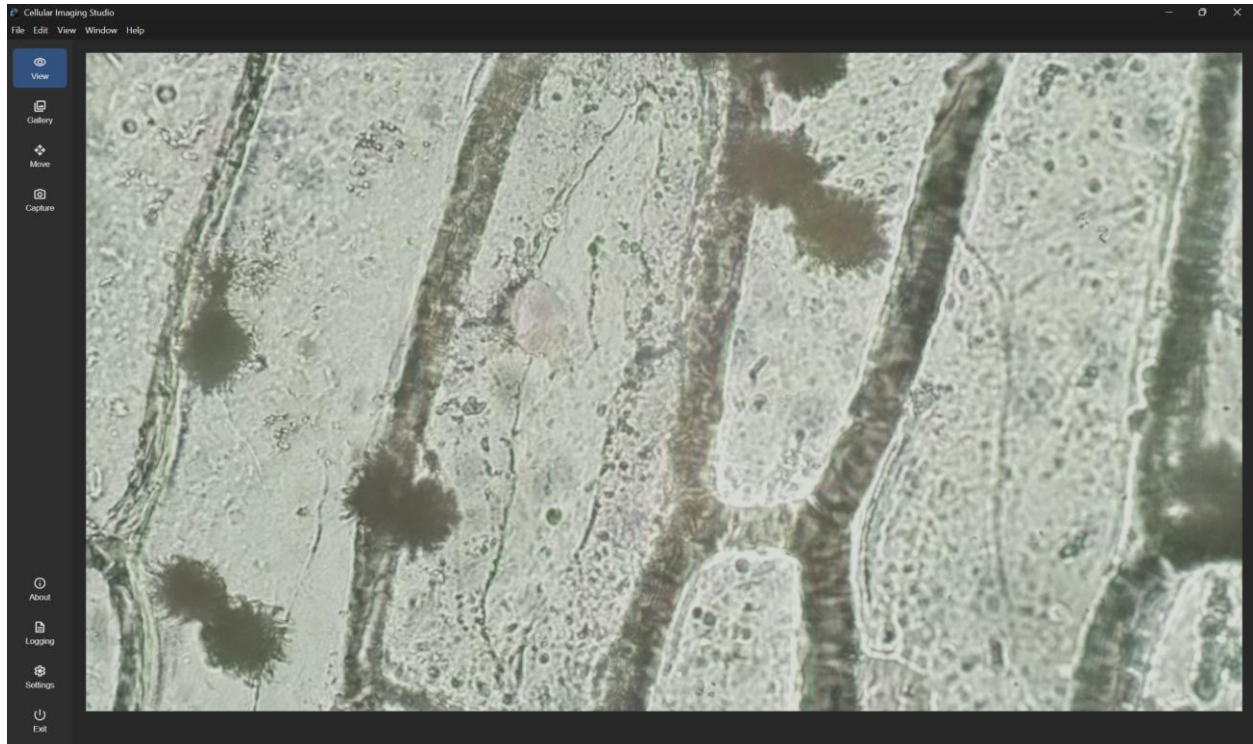
After powering up the microscope, launch the Cellular Imaging Studio program and click the **Connect** button when the start-up screen appears. After successful connection, you'll arrive at the View module.

If no microscope is detected, you'll receive an error message. [See the Troubleshooting section of this manual for additional information about connection issues.](#)

Cellular Imaging Studio Walk-Through

This section provides an overview of all the program modules and settings you'll need to capture images and videos, perform AI-driven analyses, export analysis findings, and customize the app to your liking.

View Module



The View module is the initial app landing page that provides a live preview of the sample slide loaded into the microscope. This module – and all others – includes a sidebar navigation menu to access all the program's modules and features.

TO-DO: Add a reference to troubleshooting in the case where no image appears??

Move Module

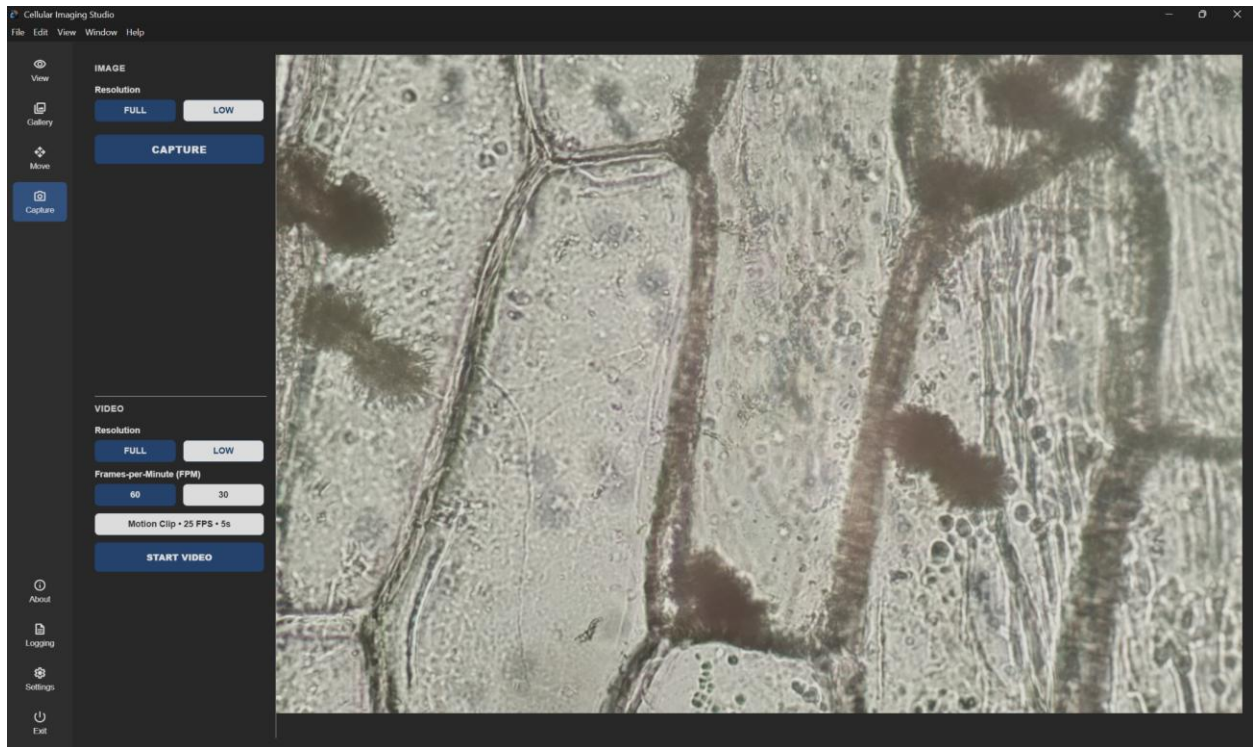


The Move module allows you to reposition the slide, using the microscope's built-in motors to shift the slide by a quantity of unit steps for each axis.

The Z-axis represents the vertical distance between the sample slide and camera, in step increments of approximately 50 nanometers. The X and Y-axis are lateral adjustments, in step increments of approximately 70 nanometers.

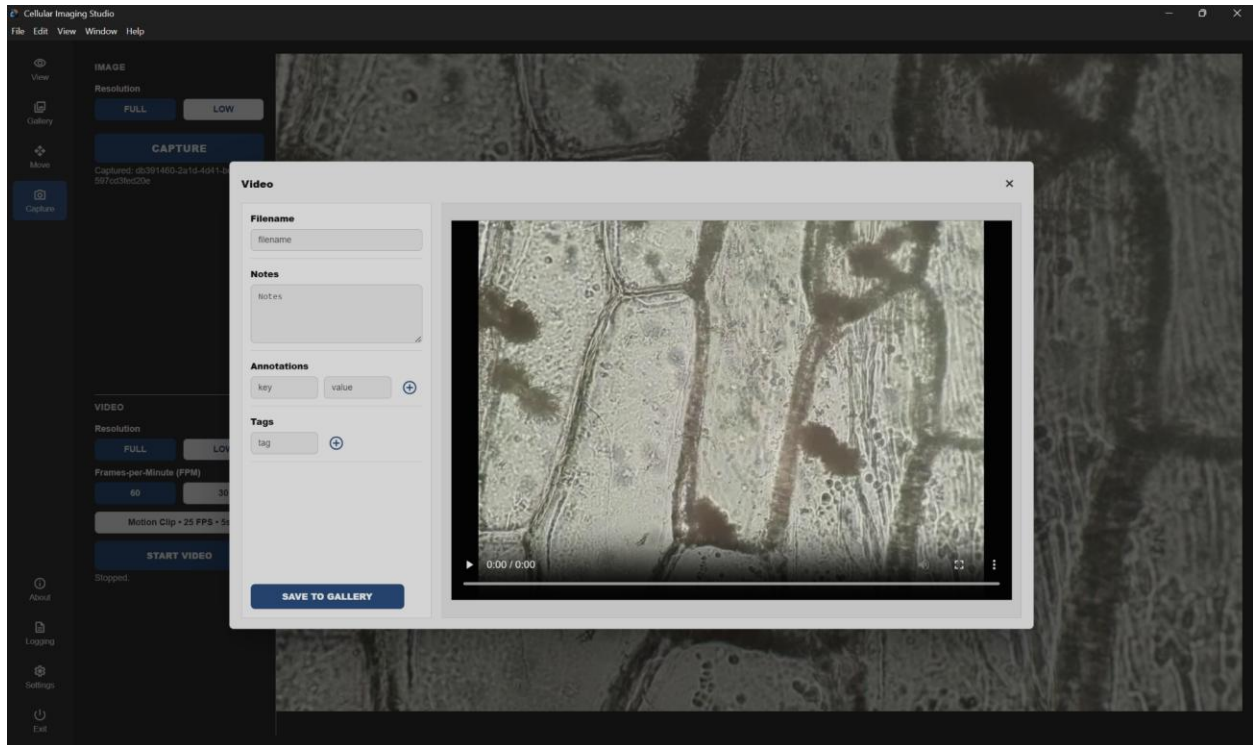
The **Center Stage** button returns the microscope to its default position of (0, 0, 0).

Capture Module



The Capture module provides you with options to capture either a still image or video clip, including resolution options for both. The video options also include setting for video duration, frame rate, and taking a time-lapse video.

Capturing an image or video is required prior to performing AI analyses.

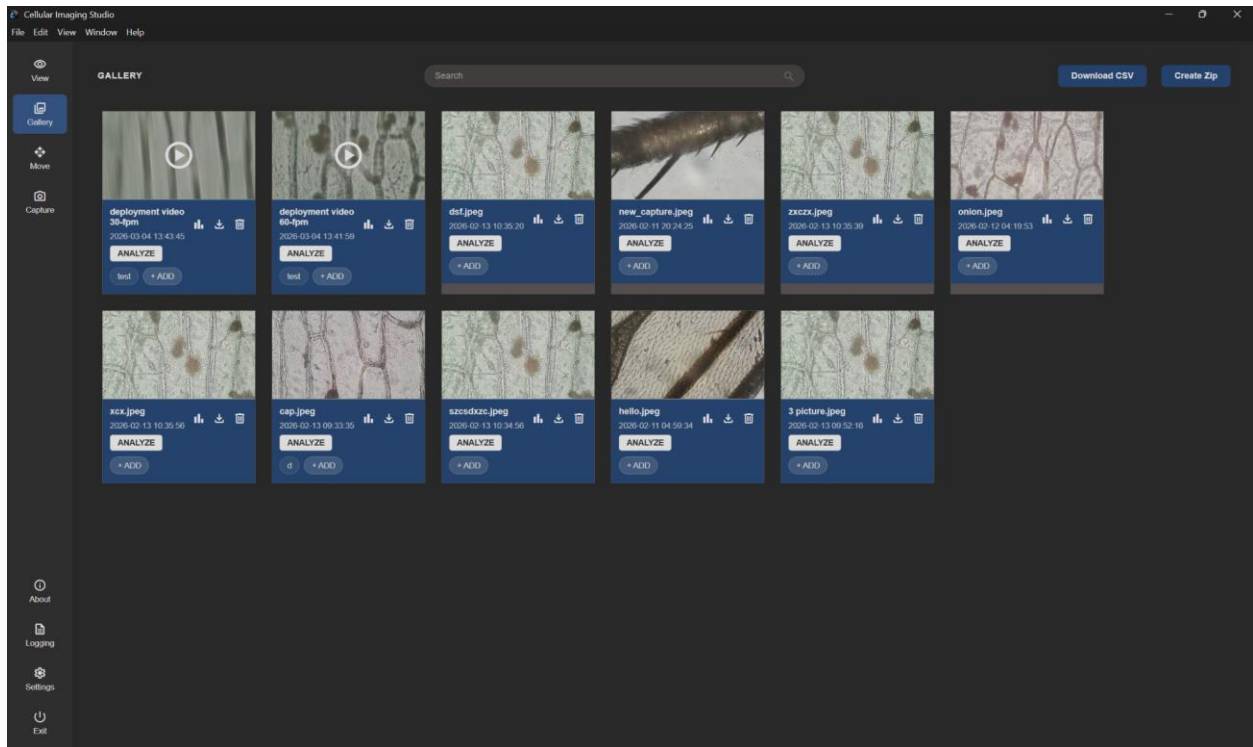


After capturing an image or recording a video, a pop-up appears, containing a preview of the captured media and options for saving it to the gallery, editing the filename, and adding any of the following optional metadata.

- Notes
- Annotations: Key + Value
- Tags

These customization options are also available for each file in the Gallery module. So, you can set them afterwards, if needed.

Gallery Module

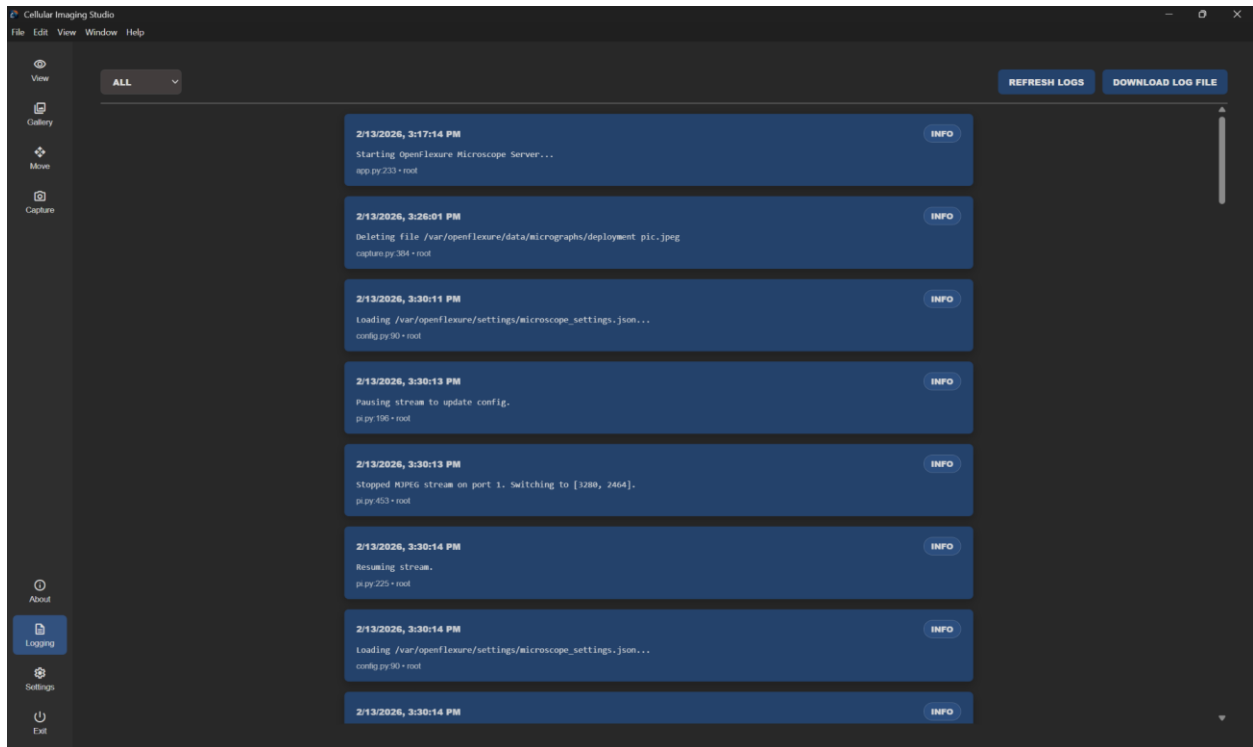


The Gallery module is a collection of all captured images and videos with options to update the metadata for each entry, several download options, and a search bar to find files by name. **The Gallery module is where you initiate our custom AI analyses.**

- **Create Zip** downloads all videos and images in a zip file.
- **Download CSV** exports a CSV file with a list of metadata of all videos and images.
- **+ADD** adds more tags or delete tags by pressing on them.
- **ANALYZE** initiates our AI cell counting and motion detection analyses.
- **Analysis Download** 📄 exports the analysis findings as a CSV file.
- **Media Download** 📄 exports a file as a JPEG image or MP4 video.
- **Delete** 🗑 permanently removes a file.

Does +ADD only include tags, or can it do notes and annotations too?
Can you update the filename from the Gallery?

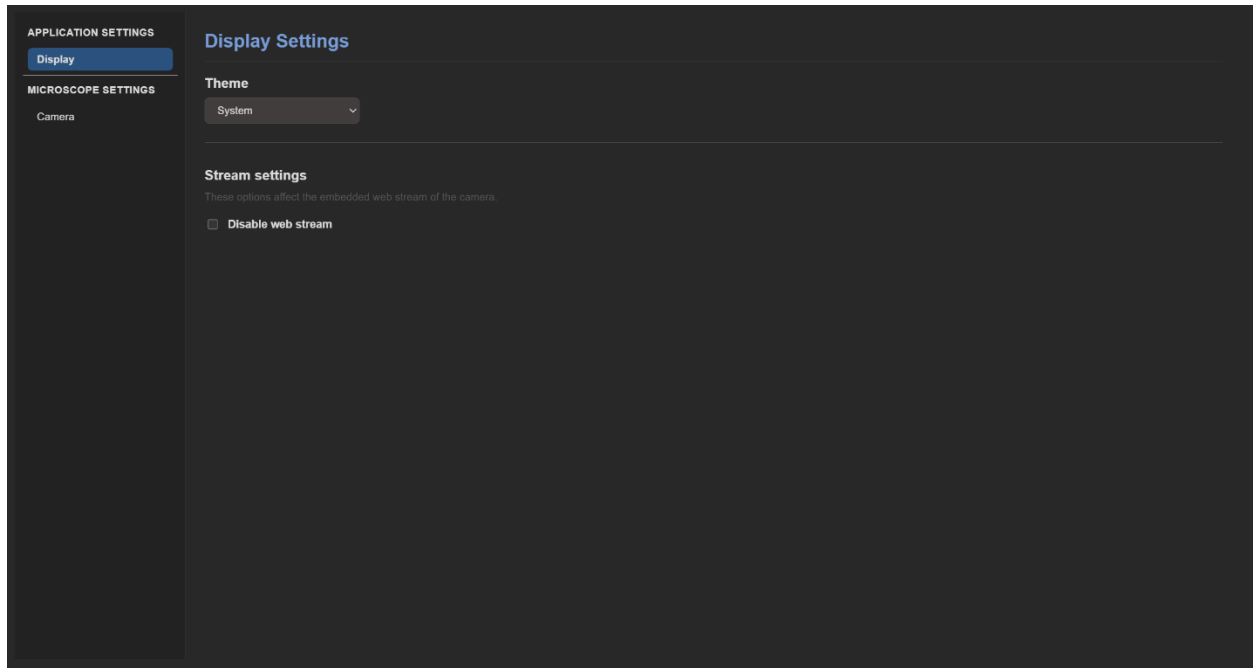
Logging Module



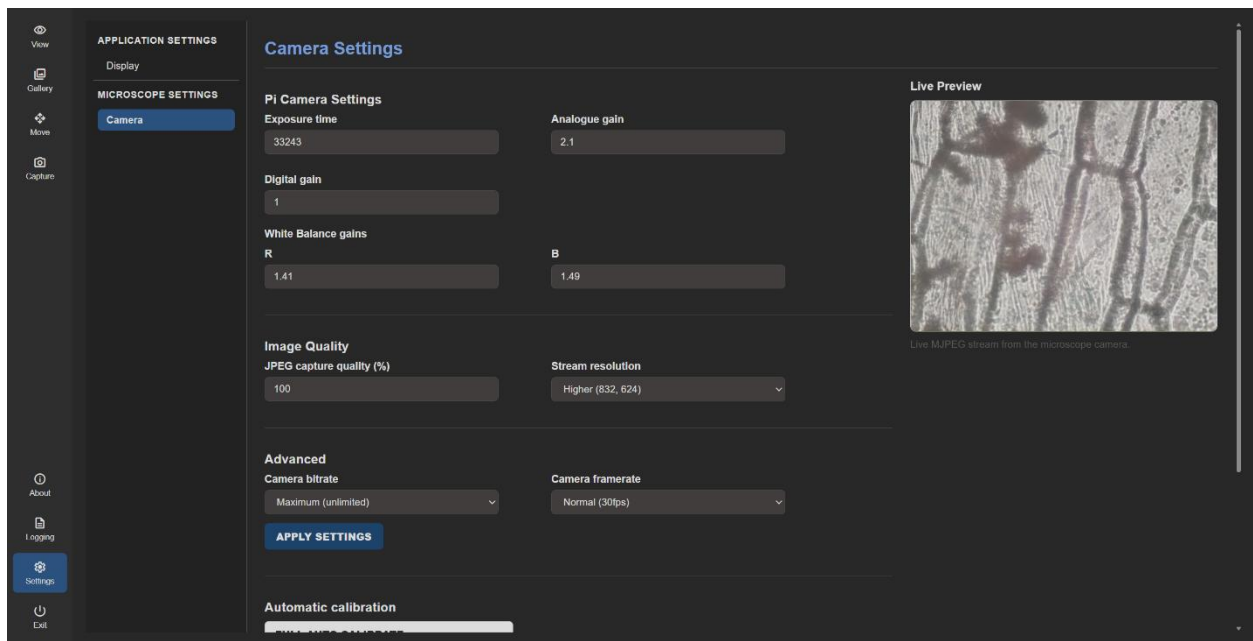
The Logging module provides access to verbose logging of all system events and relative priorities (i.e., ERROR, WARNING, and INFO). This module allows you to refresh the log, filter entries by priority, and download the entire log as a simple text file.

Settings Module

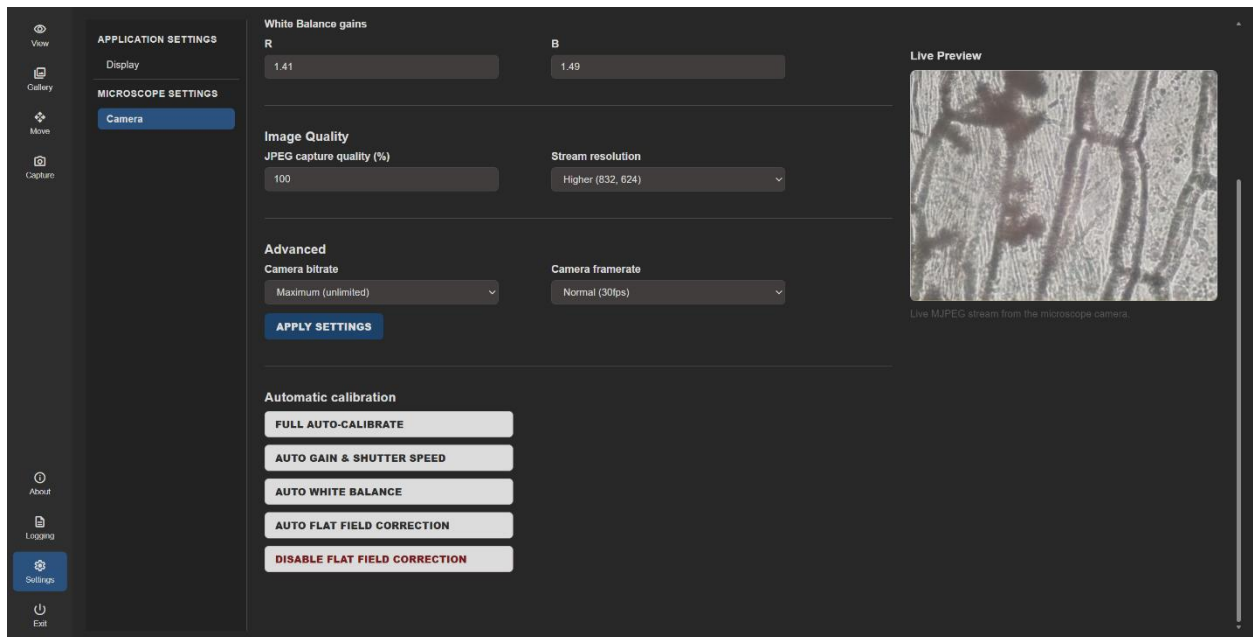
The Settings module includes two submodules, with options to adjust the UI display and update camera settings.



The Display Settings submodule includes options to toggle the UI between dark and light mode and enable/disable the camera web stream.

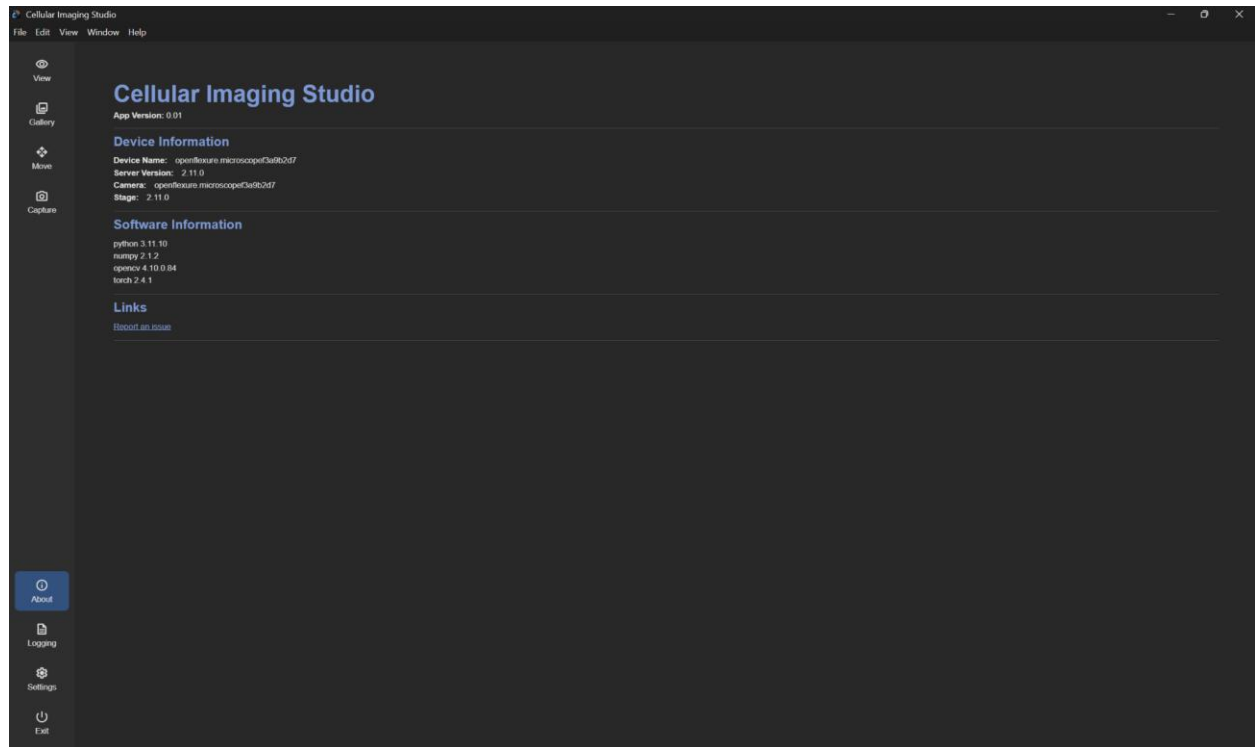


The Camera Settings submodule includes basic color correction settings and data transfer rates for streaming.



The Camera Settings submodule also includes a comprehensive auto-calibration option, automatic exposure and color correction options, and a flat-field correction to resolve imperfections generated by the camera.

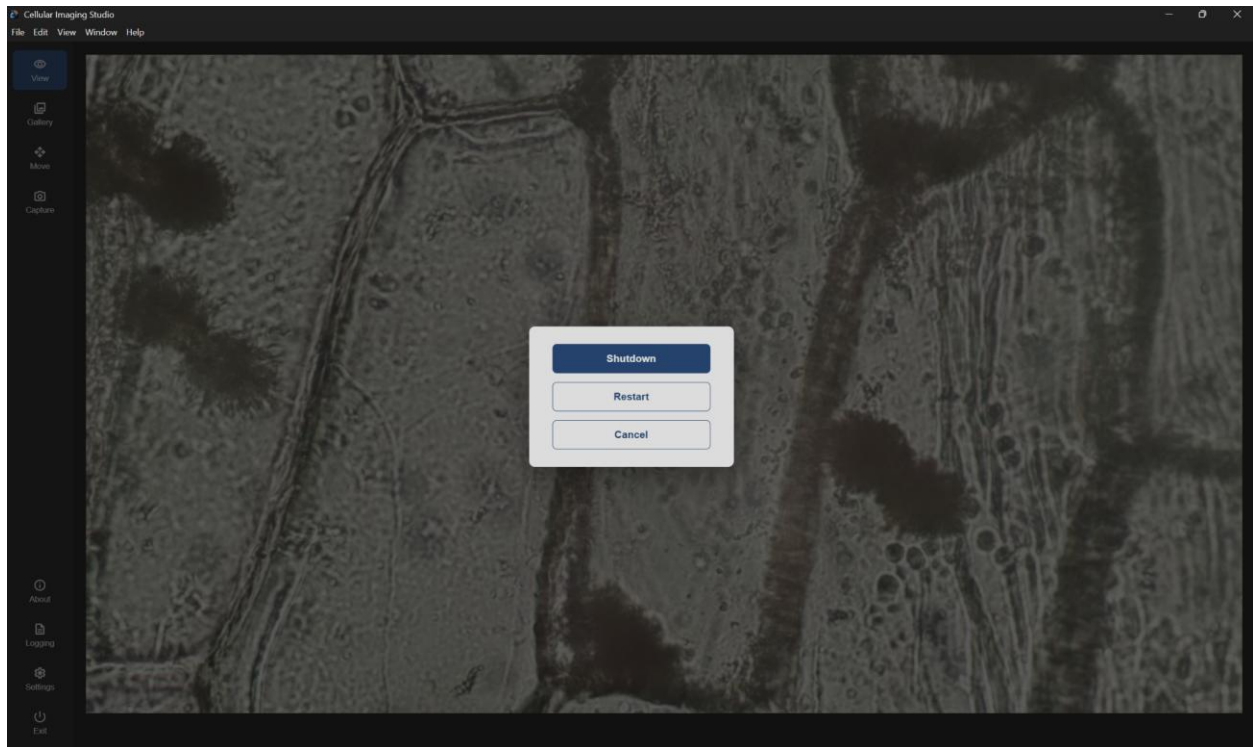
About Module



The About module provides useful backend information about the Cellular Imaging Studio app and its dependencies.

This module also includes a link for reporting issues to the development team.

Exit Module



Clicking the Exit button provides you with two options:

- **Shutdown** disconnects the microscope and terminates the program.
- **Restart** also disconnects the microscope but returns you to the initial connection screen instead of exiting the program.

Troubleshooting

TO-DO: Write three basic troubleshooting workflows.

MIT License

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Appendix: Software Architecture

Our team used the following software frameworks and libraries to develop the Cellular Imaging Studio app and foster communication between the frontend UI and Python and OpenFlexure servers.

Backend Python Libraries

- CSV: Reading and writing CSV files.
- FastAPI: Frontend UI interaction.
- FFmpeg: Video file handling.
- NumPy: Array and buffer handling for image data.
- OpenCV: Image preprocessing and segmentation for our ML/AI model.
- PathLib: Access resource directories.
- Pydantic: Request models and validation.
- Requests: OpenFlexure server API interaction and custom exception handling.
- Uvicorn: ASGI server for running the backend port.

Frontend Stack

- CSS: Styling module elements.
- Fetch API: Browser native HTTP client.
- HTML5: Markup language for module layout.
- JavaScript: Core language for UI actions.
- Node.js: JavaScript runtime environment required to run Vite and build the UI.
- npm (node package manager): Frontend dependencies.
- Vite: Dev server used for fast hot-reload development.
- Vue: Reactive JavaScript framework used to build the UI.